

DATA SCIENCE AND ANALYTICS DIVISION

ANOMALY DETECTION IN TRANSACTIONS

A Hybrid Unsupervised and Supervised Framework

Prepared by:

Titli Chanda

Roll: MDS202439

Ahana Mondal

Roll: MDS202405

May 6, 2026

1 Dataset Description

For this transaction anomaly detection project, we utilize a comprehensive structured financial ledger modeled after real-world credit card payment repositories. The dataset captures transactions made by cardholders, where anomalies represent unauthorized or fraudulent maneuvers that occur randomly across temporal segments.

1.1 Dimensionality and Features Structure

The dataset consists of a matrix comprising **5 rows** $\times$ **31 columns** at initial exploratory execution, expanding across a total population space of 284,807 unique transactions.

Due to the critical necessity of securing Personal Identifiable Information (PII) and maintaining banking confidentiality laws, the underlying features have undergone a pre-processing transformation using **Principal Component Analysis (PCA)**. The attributes are partitioned into the following structural layout:

- **Time:** A continuous numerical feature representing the exact seconds elapsed between the current transaction and the absolute first transaction initiated in the historical log.
- **V1, V2, ..., V28:** The 28 principal components generated through an orthogonal PCA mathematical extraction workflow. These represent compressed, anonymized behavioral attributes such as location coordinate maps, merchant categories, terminal specifications, and user limits.
- **Amount:** The raw, un-transformed currency value involved in the specific transaction. This metric is foundational for tracking high-value exposure and computing hour-by-hour expenditure Z-scores.
- **Class:** The distinct binary ground-truth tracking variable, where **0** represents standard authentic operations and **1** flags malicious fraudulent structural anomalies.

1.2 Exploratory Data Analysis & Class Imbalance Profile

A direct extraction of the label column distribution highlights a classic structural obstacle inherent to financial analytics: **extreme class asymmetry**. The explicit quantitative breakdown of our data vector is presented in Table 1.

Table 1: Quantitative Ground-Truth Class Distribution Breakdown

Class Label	Operational Interpretation	Absolute Count	Relative Proportion (%)
0	Normal / Authorized Transaction	284,315	99.827251%
1	Anomalous / Fraudulent Activity	492	0.172749%

Out of a total evaluated pool of **284,807 transactions**, exactly **492** instances are confirmed anomalies. This sets the base mathematical probability of any singular event being malicious at a mere **0.173%**.

This extreme data distribution renders classic accuracy metrics useless—a naive algorithm could achieve a 99.83% accuracy rating by globally predicting zero fraud. Consequently, this heavily skewed environment justifies our project’s choice to use an unsupervised Autoencoder to capture normality profiles before performing hybrid classification via an imbalance-weighted XGBoost model.

2 Initial Methodologies & Model Benchmarking

To establish an empirical baseline for transactional anomaly detection, we implemented and evaluated three distinct methodological frameworks on the test partition: a linear baseline model (**Logistic Regression**), a state-of-the-art supervised ensemble framework (**XGBoost**), and an unsupervised neural reconstruction topology (**Autoencoder**).

The test set partition maintains a consistent volume of **85,443 transactions**, containing exactly **85,295 normal instances** and **148 confirmed anomalies**.

2.1 Supervised Classifiers: Logistic Regression vs. XGBoost

Initial experimentation evaluated standard supervised classification algorithms. While global classification accuracy remains exceptionally high across both methods due to the structural data skew, their predictive integrity varies substantially when tracking individual anomaly profiles.

2.1.1 Baseline Logistic Regression

The linear baseline demonstrates a high global accuracy (99.91%) but suffers heavily from strict linear boundary limitations. It successfully surfaces 94 out of 148 actual fraud instances, but leaves 54 fraud cases completely undetected as false negatives. It balances this with a high precision (0.86), showing a lower false alarm profile at the expense of comprehensive security coverage.

2.1.2 XGBoost Ensemble Optimization

By leveraging tree-based gradient boosting alongside cost-sensitive class balancing parameters (*scale_pos_weight*), the **XGBoost** architecture provides a superior balance. It expands the true positive count significantly to **118 detected frauds**, minimizing missed fraud incidents to just 30. Although it induces a marginal increase in false alarms (35 false positives vs. Logistic Regression’s 15), its overall F1-score climbs to a robust **0.78**.

2.2 Unsupervised Approach: Autoencoder Reconstruction Deviations

Recognizing that fraud patterns continuously mutate over time, we deployed a deep structural Autoencoder. This network was trained exclusively on confirmed normal transactions ($Class = 0$) to map out standard transactional boundaries.

When evaluated on the mixed test matrix, anomalies are identified based on elevated mean squared reconstruction errors. The operational results highlight a stark trade-off:

- **High Anomaly Recall (0.84):** The unsupervised framework captures a vast majority of fraud variants, successfully surfacing **125 out of 148 actual frauds**—the highest baseline recall achieved among all three standard configurations.
- **Severe Precision Drop (0.02):** Because the network lacks explicit labels to learn precise fraudulent decision boundaries, it generates massive false-alarm clusters. It flags **5,091 legitimate transactions as anomalies**, causing a drop in precision to 2%, which would overwhelm real-world fraud operational teams.

2.3 Comparative Performance Summary

The complete quantitative metrics extracted from the test matrix tracking profiles across all three paradigms are compiled in Table 2 and Table 3.

Table 2: Comparative Matrix: Performance Metrics Across Evaluated Architectures

Methodological Strategy	Global Accuracy	Class 1 Precision	Class 1 Recall	Class 1 F1-Score
Logistic Regression	99.919%	0.86	0.64	0.73
XGBoost Classifier	99.924%	0.77	0.80	0.78
Unsupervised Autoencoder	94.015%	0.02	0.84	0.05

Table 3: Empirical Confusion Matrix Comparison Log across Baseline Models

	Logistic Regression		XGBoost Ensemble		Autoencoder Network	
	<i>Pred 0</i>	<i>Pred 1</i>	<i>Pred 0</i>	<i>Pred 1</i>	<i>Pred 0</i>	<i>Pred 1</i>
Actual Normal (0)	85,280	15	85,260	35	80,204	5,091
(Support: 85,295)						
Actual Anomaly (1)	54	94	30	118	23	125
(Support: 148)						

2.4 Justification for the Proposed Hybrid Architecture

The empirical strengths and limitations of these initial methodologies directly validate the advanced hybrid multi-stage pipeline designed later in this project:

1. The **Autoencoder** possesses an excellent capacity to flag hidden anomalies (125 caught) but cannot isolate true fraud independently without causing excessive false alarms.
2. The **XGBoost** engine provides precise decision structures but relies solely on raw features.

By routing the reconstruction errors generated by the Autoencoder and density maps from K-Means directly as *engineered features* into a downstream XGBoost classifier, our hybrid framework leverages the high recall of deep unsupervised models while preserving the high precision control of tree-based supervised ensembles.

3 Proposed Methodology: The Hybrid Multi-Stage Pipeline

To resolve the high false-alarm vulnerabilities observed in purely unsupervised frameworks, while keeping the high pattern recognition of deep architectures, we implement a multi-stage hybrid inference pipeline.

Instead of relying solely on raw anonymized PCA vectors (V_1 to V_{28}), this system functions as an algorithmic assembly line. It uses real-time temporal analytics, clustering density mappings, and neural reconstruction deviations to build a high-dimensional feature matrix for a downstream cost-weighted XGBoost model.

3.1 Generation and Simulation of Streaming Data

In actual production environments, transactions do not arrive in static batches; they flow sequentially as distinct temporal records. To evaluate our models under production conditions, we construct a **Streaming Simulation Engine**.

The dataset is first sorted strictly by its temporal sequence using the *Time* axis to preserve genuine chronological trends. We apply an 80/20 chronological split: the first 80% of historical logs form our static training environment, while the remaining 20% are set aside to simulate a real-time stream.

During the testing phase, an iterative loop goes through this streaming partition row-by-row. This process isolates each individual transaction array, preventing data leakage from subsequent records. This setup perfectly replicates a live banking ledger where each event must be approved or blocked immediately upon arrival.

3.2 Model Selection Strategy

The core predictive engine uses an optimized **XGBoost (Extreme Gradient Boosting)** tree ensemble. XGBoost was selected for several reasons:

- **Non-Linear Decision Boundaries:** Unlike linear models, it excels at identifying complex interactions between raw values and our engineered meta-features.
- **Imbalance Mitigation:** By utilizing the *scale_pos_weight* parameter, it penalizes the misclassification of the rare minority fraud class during tree construction, adjusting for the baseline 0.173% fraud distribution.
- **Inference Speed:** Once trained, the decision trees can evaluate a feature vector in fractions of a millisecond, fitting well within strict financial transaction latency budgets.

3.3 Detailed Breakdown of Engineered Features

Before data enters the model, it is enriched with specialized features designed to capture specific fraudulent behaviors:

3.3.1 Temporal Delta Tracking ($\Delta Time$)

Fraudulent programmatic attacks often execute multiple transactions in seconds to drain an account before a block is triggered. The pipeline tracks internal state memory by caching the timestamp of the preceding event (t_{i-1}) and computing the operational gap:

$$\Delta Time = t_i - t_{i-1} \quad (1)$$

Extremely small values for $\Delta Time$ suggest automated machine behavior.

3.3.2 Global Transaction Velocity

A continuous rolling window counts total transaction frequencies within the immediate historical time cluster. This helps catch sudden spikes in system-wide or user-specific activities, which often point to coordinated card-testing scams.

3.3.3 Hour of Day Discretization

Human spending habits change throughout the day. To map raw continuous seconds into standard human activity cycles, timestamps are converted into discrete 24-hour cycles using modulo operations:

$$Hour = (\lfloor Time \rfloor \div 3600) \pmod{24} \quad (2)$$

3.3.4 Contextual Value Rarity ($Amount_ZScore$)

A \$500 transaction at 2:00 PM is normal, but the same amount at 3:00 AM can be highly anomalous. This feature standardizes transaction values relative to changing hourly baselines. The system pre-aggregates the historical mean (μ_{hour}) and standard deviation (σ_{hour}) for each hour slot, transforming the incoming amount via:

$$Amount_ZScore = \frac{Amount - \mu_{hour}}{\sigma_{hour}} \quad (3)$$

3.3.5 Cluster Proximity and Fraud Density

The scaled vector is mapped into $K = 10$ distinct spaces via K-Means. It generates two features: *Cluster Proximity Distance* (the Euclidean distance to the nearest geometric

cluster center, which catches spatial outliers) and *Historical Cluster Fraud Density* (which checks the historical fraud probability of that specific cluster).

3.3.6 Autoencoder Reconstruction Error (MSE)

An Autoencoder neural network processes the vector, compressing it down to a tight bottleneck layer before reconstructing it. Because its weights are optimized solely on authorized data, it cannot properly reconstruct abnormal fraud patterns. This structural mismatch is extracted as a feature using Mean Squared Error (MSE):

$$MSE = \frac{1}{N} \sum_{j=1}^N \left(x_{scaled}^{(j)} - \hat{x}_{scaled}^{(j)} \right)^2 \quad (4)$$

Table 4: Structure of the Final Concatenated Feature Vector (X_{final})

Raw Time & Amount (2 Features)	PCA Subspace (V_1 to V_{28})	Engineered Context ($\Delta Time$, Velocity, Hour, Z)	Neural Error (MSE)	Cluster Den (Historical R
-----------------------------------	---------------------------------------	---	---------------------------	------------------------------

3.4 Microservice Architecture via API Communication

To move this pipeline from a local research notebook to an accessible production service, we wrapped the entire inference architecture in a **FastAPI** web service framework.

The communication pipeline operates through structured asynchronous network interactions:

1. **Client Payload Initiation:** An external payment gateway securely packages a transaction’s raw parameters as a JSON list and transmits an HTTP POST request to the server’s dedicated endpoint: `http://127.0.0.1:8000/predict`.
2. **Pydantic Schema Guardrails:** The receiving API uses a strict Pydantic parsing class to check the type and length of the payload. If a request has missing values or the incorrect format, it triggers an immediate HTTP 400 `Bad Request` error to protect the internal models.
3. **Asynchronous Computation and Response:** If the format is correct, the raw numbers pass through our engineering, scaling, and multi-model feature generation stages. The final vector is evaluated by the XGBoost classifier, and the server returns a clean JSON response:

```
{"fraud": 1, "status": "BLOCKED"}
```


This entire request-to-verdict lifecycle completes in milliseconds, providing an efficient balance of security and speed.

4 Conclusion and Future Directions

In this project, we designed, implemented, and evaluated a robust, production-ready hybrid machine learning pipeline for real-time transactional anomaly detection. Navigating an environment with extreme class asymmetry (0.173% fraud density), we demonstrated that relying purely on standalone supervised or unsupervised methods forces an unacceptable operational trade-off between missing actual fraud and generating excessive false alarms.

By structuring a multi-stage architecture, we turned these individual model limitations into engineering strengths. The deep bottleneck Autoencoder acts as a highly sensitive structural filter, catching hidden anomaly variants through reconstruction deviations (MSE). Simultaneously, the K-Means clustering algorithm maps out spatial risk profiles and historical cluster fraud density. Combining these unsupervised features with real-time temporal and velocity indicators allowed us to build an enriched, high-dimensional feature space. This composite matrix enables the downstream, cost-weighted XGBoost classifier to make highly accurate decisions with minimal false positives.

Furthermore, wrapping this analytical pipeline within a lightweight, asynchronous FastAPI microservice container shifts this project from a theoretical data science experiment into a practical backend solution. The system enforces strict Pydantic input schema validation boundaries and processes incoming real-time transactions sequentially in milliseconds. This achieves an optimal balance between rigorous transaction security and low-latency user experiences.

4.1 Future Extensions

While the current framework provides excellent protection and low-latency execution, several key enhancements are planned for future versions:

1. **State Memory Migration to Redis:** Migrating the in-memory state dictionary to a dedicated, high-speed Redis caching database layer will ensure accurate calculations of rolling time velocity and contextual Z -scores across multi-server distributed production setups.
2. **Dynamic Concept Drift Tracking:** Implementing automated drift monitoring (such as Population Stability Index tracking) will alert operations when user spending habits or fraud tactics shift, preventing performance degradation over time.
3. **Graph Neural Network (GNN) Integration:** Moving beyond isolated transaction vectors, future iterations will explore structural relationship networks using

Graph Neural Networks to flag complex money laundering chains and coordinated syndicate fraud rings.